That's enough examples for now; instead, let's look at extracting some guidelines for writing lock-free data structures from the examples.

7.3. Guidelines for writing lock-free data structures

If you've followed through all the examples in this chapter, you'll appreciate the complexities involved in getting lock-free code right. If you're going to design your own data structures, it helps to have some guidelines to focus on. The general guidelines regarding concurrent data structures from the beginning of chapter 6 still apply, but you need more than that. I've pulled a few useful guidelines out from the examples, which you can then refer to when designing your own lock-free data structures.

7.3.1. Guideline: use std::memory_order_seq_cst for prototyping

`std::memory_order_seq_cst` is much easier to reason about than any other memory ordering because all such operations form a total order. In all the examples in this chapter, you've started with `std::memory_order_seq_cst` and only relaxed the memory-ordering constraints once the basic operations were working. In this sense, using other memory orderings is an *optimization*, and as such you need to avoid doing it prematurely. In general, you can only determine which operations can be relaxed when you can see the full set of code that can operate on the guts of the data structure. Attempting to do otherwise just makes your life harder. This is complicated by the fact that the code may work when tested but isn't guaranteed. Unless you have an algorithm checker that can systematically test all possible combinations of thread visibilities that are consistent with the specified ordering guarantees (and such things do exist), just running the code isn't enough.

7.3.2. Guideline: use a lock-free memory reclamation scheme

One of the biggest difficulties with lock-free code is managing memory. It's essential to avoid deleting objects when other threads might still have references to them, but you still want to delete the object as soon as possible